Raspberry PI Pico com MicroPython

Fernando Anselmo
http://fernandoanselmo.orgfree.com/wordpress/

Versão 1.1 em 10 de janeiro de 2026

Resumo

Raspberry Pi Pico é uma placa com microcontrolador de baixo custo e alto desempenho que se destaca por seu tamanho reduzido e por oferecer uma alternativa acessível para projetos de hardware personalizados. É o primeiro microcontrolador desenvolvido pela Raspberry Pi Foundation e é baseado no chip RP2040, projetado pela própria fundação. A adoção do MicroPython no Raspberry Pi Pico apresenta várias vantagens, como a facilidade de programação e a rapidez no desenvolvimento de protótipos, devido à sintaxe concisa e legibilidade do Python.

1 Introdução

Uma das principais vantagens de utilizar MicroPython para programar o Raspberry Pi Pico é a capacidade de executar comandos interativos em tempo real por meio do REPL (Read-Eval-Print Loop), sendo possível testar códigos e funções instantaneamente. Além disso, a linguagem Python possui uma vasta coleção de bibliotecas e módulos que podem ser facilmente integrados aos projetos, o que expande consideravelmente as possibilidades de uso do Pico em uma variedade de aplicações, incluindo automação residencial, robótica, sensores e muito mais.

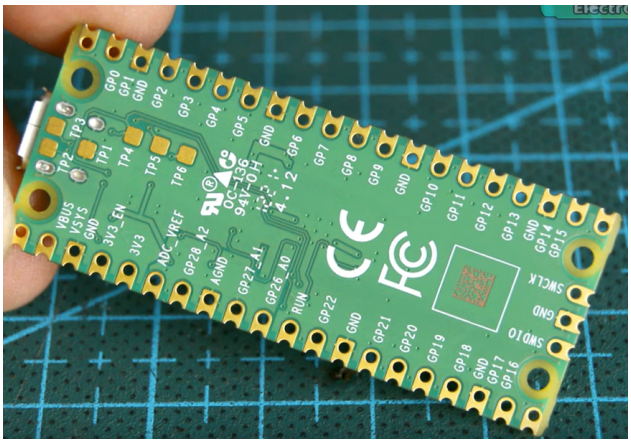


Figura 1: *Raspberry Pi Pico*

Outro ponto significativo é a facilidade com que novos usuários e programadores podem começar a usar o MicroPython no Raspberry Pi Pico. A linguagem é conhecida por ser acessível a iniciantes, tornando a curva de aprendizado mais suave em comparação com outras linguagens mais complexas. Isso se traduz em uma rampa mais rápida para a criação de projetos eletrônicos e de programação, tornando o Pico uma opção atraente para educação, hobbistas e até mesmo para desenvolvedores de protótipos profissionais.

Em termos de recursos de hardware, o Raspberry Pi Pico oferece interfaces digitais flexíveis, permitindo a conexão com uma variedade de dispositivos e sensores externos. Sua arquitetura inclui uma configuração de memória generosa para um microcontrolador e vem com suporte tanto para MicroPython quanto para C/C++, garantindo versatilidade e desempenho adequados para uma gama de projetos de microcontroladores. Com essas características, o Raspberry Pi Pico com MicroPython representa uma combinação poderosa para a criação de projetos inovadores e complexos, mantendo um nível de simplicidade e acessibilidade para todos os níveis de habilidade.

2 Projetos para a prática

Aprender a programar com MicroPython por meio de exemplos práticos traz vantagens significativas, especialmente para aqueles que estão começando ou deseja ver resultados rápidos. Exemplificando, imagine que um estudante ou entusiasta queira automatizar uma estufa. Com MicroPython, seria possível programar o Raspberry Pi Pico para controlar a umidade e a temperatura, lendo dados de sensores em tempo real e ajustando um sistema de irrigação ou de controle de clima automaticamente. A implementação desse tipo de projeto ganha em simplicidade e clareza com a concisão da linguagem Python, e a imediata retroalimentação fornecida pelo REPL do MicroPython facilita a depuração e o aperfeiçoamento contínuo do código.

Outra vantagem prática de aprender MicroPython se revela na velocidade com que se pode passar da ideia ao protótipo funcional. Por exemplo, um projeto de robótica educacional onde os alunos estejam construindo um pequeno robô móvel. O uso do MicroPython permite aos alunos focar nos conceitos de lógica de programação e interação com componentes de hardware sem a sobrecarga de linguagens mais complexas ou de baixo nível.

Isso significa que podemos dedicar mais tempo a entender como os motores funcionam e como controlá-los, ou como utilizar sensores para que o robô reaja ao ambiente. Assim, o MicroPython torna-se um facilitador de aprendizado e inovação, permitindo uma transição quase direta entre a teoria e a prática.

ATENÇÃO: Em todos os projetos descritos utilizaremos uma placa de prototipagem, *jumpers* - fios para conexão (quantidade conforme vista no diagrama dos projetos) e um Raspberry PI Pico já configurado e iniciado para aceitar a programação MicroPython (veja diversos tutoriais e vídeos na Internet que ensinam como realizar este processo), que não serão descritos nos materiais necessários.

3 Projetos com LEDs

3.1 Controlar um LED

Este projeto possui os seguintes materiais:

- LED vermelho

- Resistor de 220 Ω

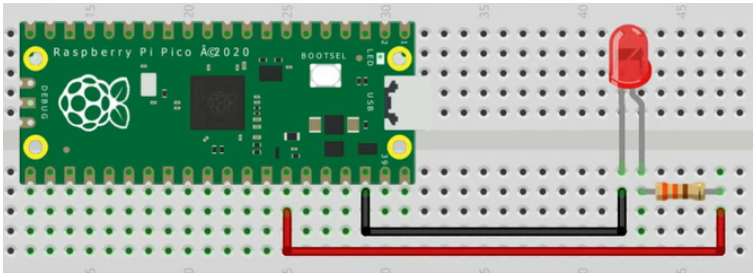


Figura 2: *Esquemático para o Projeto*

Ligações dos *jumpers*:

- VERMELHO: para a placa na GP28
- PRETO: para a placa na GND

```
1  from machine import Pin
2  import utime
3
4  led1 = Pin(28, Pin.OUT)
5  delay = .40
6  while True:
7     led1.value(1)
8     utime.sleep(delay)
9     led1.value(0)
10    utime.sleep(delay)
```

3.2 Controlar vários LEDs

Este projeto possui os seguintes materiais:

- 5 LEDs vermelho
- 5 Resistores de 220 Ω

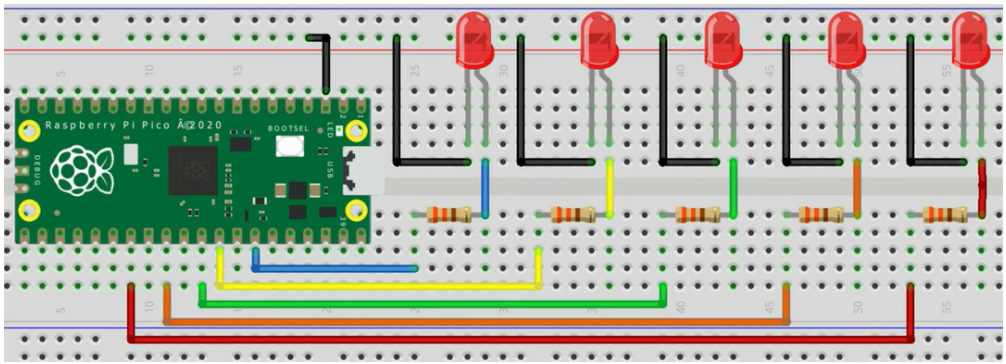


Figura 3: *Esquemático para o Projeto*

Ligações dos *jumpers*:

- AZUL: para a placa na GP28
- AMARELO: para a placa na GP27
- VERDE: para a placa na GP26
- LARANJA: para a placa na GP22
- VERMELHO: para a placa na GP21
- PRETO: para a placa na GND

```
from machine import Pin
import utime

led1 = Pin(28, Pin.OUT)
led2 = Pin(27, Pin.OUT)
led3 = Pin(26, Pin.OUT)
led4 = Pin(22, Pin.OUT)
led5 = Pin(21, Pin.OUT)
delay = .40

def selecionar(l1, l2, l3, l4, l5):
  led1.value(l1)
  led2.value(l2)
  led3.value(l3)
  led4.value(l4)
  led5.value(l5)
  utime.sleep(delay)

while True:
  selecionar(1,0,0,0,0)
  selecionar(0,1,0,0,0)
  selecionar(0,0,1,0,0)
  selecionar(0,0,0,1,0)
  selecionar(0,0,0,0,1)
```

3.3 Controlar LED com PushButton

Este projeto possui os seguintes materiais:

- LED vermelho
- PushButton
- Resistor de 220 Ω

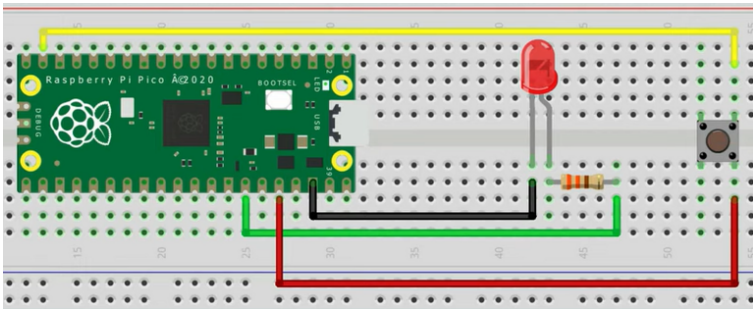


Figura 4: *Esquemático para o Projeto*

Ligações dos *jumpers*:

- AMARELO: Lado A do Pushbutton para a placa na GP14
- VERMELHO: Lado B do Pushbutton para a placa em 3.3v
- VERDE: para a placa na GP28
- PRETO: para a placa na GND

```
1  from machine import Pin
2  import time
3
4  led = Pin(28, Pin.OUT)
5  pbutton = Pin(14, Pin.IN, Pin.PULL_DOWN)
6
7  while True:
8     if pbutton.value():
9        led.toggle()
10       time.sleep(0.5)
```

3.4 LED RGB

Este projeto possui os seguintes materiais:

- LED RGB
- 3 resistores de 220 Ω

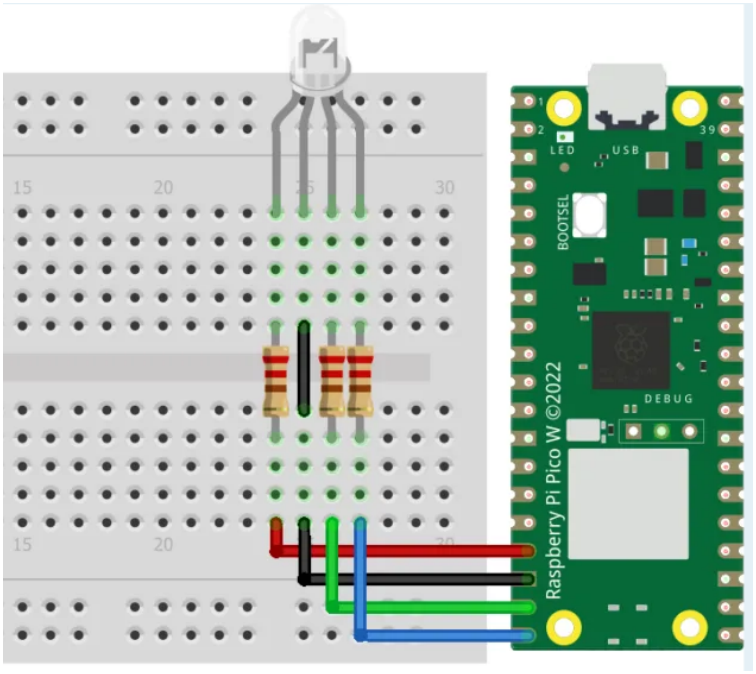


Figura 5: *Esquemático para o Projeto*

```
from machine import Pin,PWM
import utime
import random

Led_R = PWM(Pin(13))
Led_G = PWM(Pin(14))
Led_B = PWM(Pin(15))

# Define a frequencia
Led_R.freq(2000)
Led_G.freq(2000)
Led_B.freq(2000)

while True:
  # Cria valores aleatorios
  R = random.randint(0,65535)
  G = random.randint(0,65535)
  B = random.randint(0,65535)
  print(R,G,B)
  Led_R.duty_u16(R)
  Led_G.duty_u16(G)
  Led_B.duty_u16(B)
  utime.sleep_ms(100)
```

3.5 Ativar/Desativar LED RGB com Som

Este projeto possui os seguintes materiais:

- Módulo LED RGB
- Módulo de Sensor de Som

ATENÇÃO Ao usar o módulo os resistores já estão embutidos, caso use o LED RGB utilize, conforme o projeto anterior os resistores de 220 Ω.

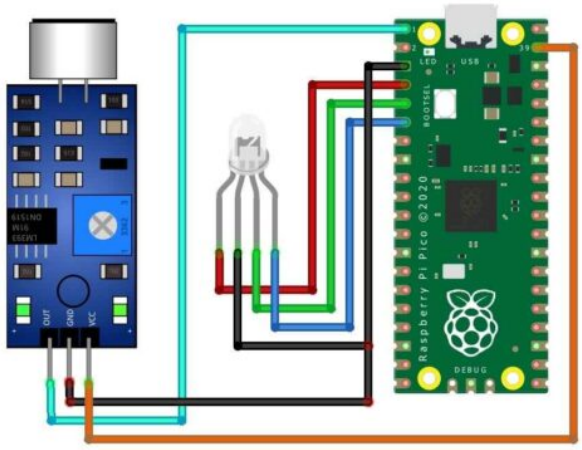


Figura 6: *Esquemático para o Projeto*

```
from machine import Pin,PWM
from  utime  import sleep_ms

# Porta interna pull-down
sound = Pin(0, Pin.IN, Pin.PULL_DOWN)
Led_R = PWM(Pin(2))
Led_G = PWM(Pin(3))
Led_B = PWM(Pin(4))
Led_R.freq(2000)
Led_G.freq(2000)
Led_B.freq(2000)

while True:
  print(sound.value())
  if sound.value() == 1:
    Led_R.duty_u16(65535)
    Led_G.duty_u16(65535)
    Led_B.duty_u16(65535)
    sleep_ms(2000)
  else:
    Led_R.duty_u16(0)
    Led_G.duty_u16(0)
    Led_B.duty_u16(0)
```

3.6 Semáforo de Trânsito

Este projeto possui os seguintes materiais:

- LED vermelho
- LED verde
- LED amarelo
- 3 Resistores de 220 Ω

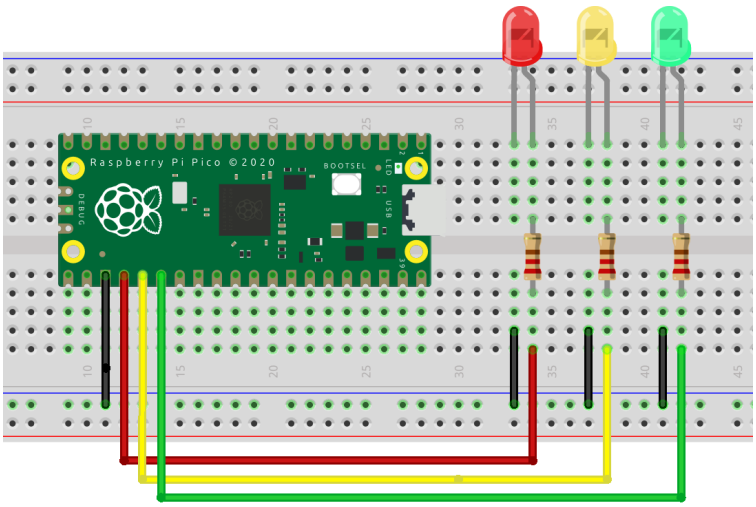


Figura 7: *Esquemático para o Projeto*

Ligações dos *jumpers*:

- VERMELHO: para a placa na GP2
- AMARELO: para a placa na GP3
- VERDE: para a placa na GP4
- PRETO: para a placa na GND

```
import machine
import time

# Define os pinos
PIN_RED = 2
PIN_YELLOW = 3
PIN_GREEN = 4

# Marcar como saida
red = machine.Pin(PIN_RED, machine.Pin.OUT)
yellow = machine.Pin(PIN_YELLOW, machine.Pin.OUT)
green = machine.Pin(PIN_GREEN, machine.Pin.OUT)

# Garantir que o amarelo esta desligado
yellow.value(0)

while True:
  # Ligar o verde
  red.value(0)    # desligar vermelho
  green.value(1)  # ligar verde
  time.sleep(5)   # Esperar 5 segs

  # Ligar o amarelo
  green.value(1)  # desligar verde
  yellow.value(1) # ligar amarelo
  time.sleep(3)   # Esperar 3 segs

  # Ligar o vermelho
```

```
29  yellow.value(1) # desligar amarelo
30  red.value(1)    # ligar vermelho
31  time.sleep(10)  # Esperar 10 segs
```

DICA: Como exercício troque os 3 LEDs por um único LED RGB. **DICA 2**: Outra atividade bem interessante é fazer com que o sinal mude apenas após pressionarmos um **pushbutton**.

4 Projetos com Telas

4.1 Display OLED

Este projeto possui os seguintes materiais:

- Módulo display OLED LCD 0,96"(SSD1306)

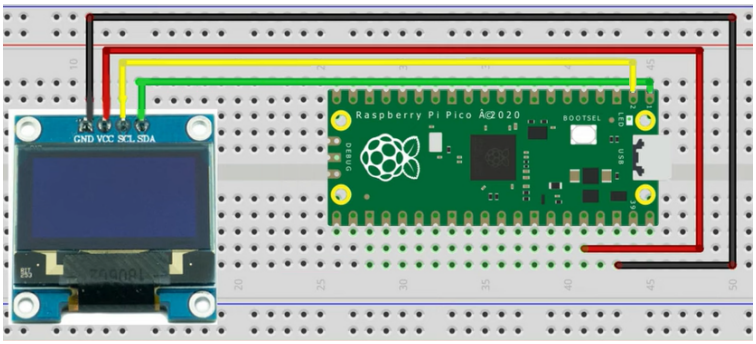


Figura 8: *Esquemático para o Projeto*

Ligações dos *jumpers*:

- VERDE: SDA do OLED para a placa na GP0
- AMARELO: SCL do OLED para a placa na GP1
- VERMELHO: VCC do OLED para a placa em 3.3v
- PRETO: GND do OLED para a placa na GND

Nas opções do Thonny (Tools — Options... — Interpreter), verificar se está marcado "MicroPython (Raspberry Pi Pico)

Baixar as bibliotecas (Tools — Manage packages...):

- micropython-ssd1306
- micropython-oled

```
1 from machine import Pin, I2C
2 from ssd1306 import SSD1306_I2C
3 from oled import Write, GFX, SSD1306_I2C
```

```
from oled.fonts import ubuntu_mono_15, ubuntu_mono_20

WIDTH = 128
HEIGHT = 64

i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
oled = SSD1306_I2C(WIDTH, HEIGHT, i2c)

write15 = Write(oled, ubuntu_mono_15)
write20 = Write(oled, ubuntu_mono_20)

write20.text("OLED", 0, 0)
write15.text("Hello", 0, 20)
oled.text("Raspberry Pi Pico", 0, 40)
oled.show()
```

4.2 Display OLED com Potenciômetro

Este projeto possui os seguintes materiais:

- Módulo display OLED LCD 0,96”(SSD1306)
- Potenciômetro de 10 KΩ

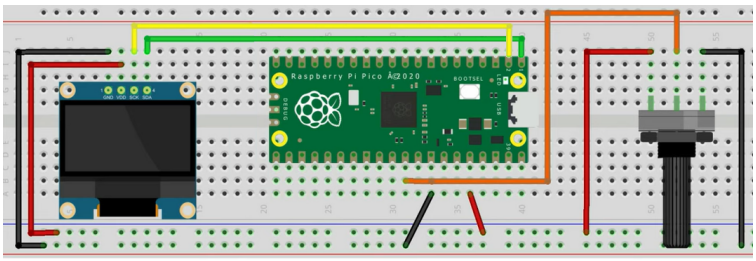


Figura 9: *Esquemático para o Projeto*

Ligações dos *jumpers*:

- PRETO: primeira perna do potenciômetro para a placa na GND
- LARANJA: segunda perna do potenciômetro para a placa na GP26
- VERMELHO: terceira perna do potenciômetro para a placa em 3.3v
- VERDE: SDA do OLED para a placa na GP0
- AMARELO: SCL do OLED para a placa na GP1
- VERMELHO: VCC do OLED para a placa em 3.3v
- PRETO do GND do OLED para a placa na GND

```
from machine import Pin, I2C
from ssd1306 import SSD1306_I2C
from oled import Write, GFX, SSD1306_I2C
from oled.fonts import ubuntu_mono_15, ubuntu_mono_20
```

```
5
6  WIDTH = 128
7  HEIGHT = 64
8
9  i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
10 oled = SSD1306_I2C(WIDTH, HEIGHT, i2c)
11
12 pot_val = machine.ADC(26)
13 fator_conversao = 3.3 / 65535
14
15 write15 = Write(oled, ubuntu_mono_15)
16 write20 = Write(oled, ubuntu_mono_20)
17
18 while True:
19    dado = pot_val.read_u16() * fator_conversao
20    oled.fill(0)
21    write20.text("Valor: ", 0, 0)
22    write15.text(str(round(dado,1)), 0, 20)
23    oled.show()
```

4.3 Ver a Distância no Display OLED

Este projeto possui os seguintes materiais:

- Módulo display OLED LCD 0,96”(SSD1306)
- Módulo Sensor Ultrassônico (HC-SR04)

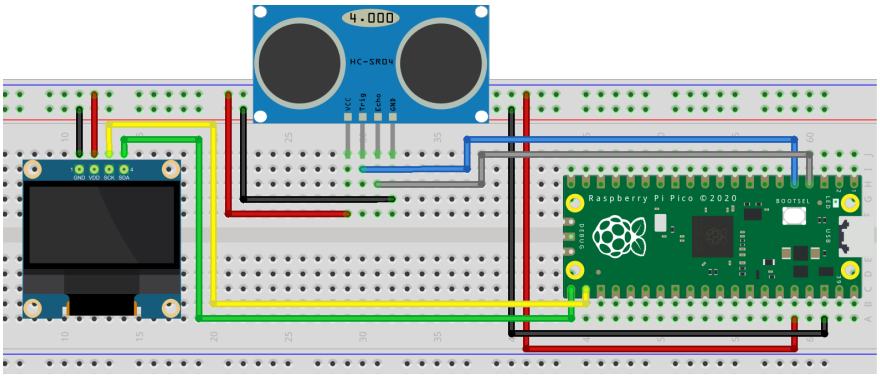


Figura 10: *Esquemático para o Projeto*

Ligações dos *jumpers*:

- AZUL: Trigger do HC para a placa na GP13
- CINZA: ECHO do HC para a placa na GP2
- VERDE: SDA do OLED para a placa na GP0
- AMARELO: SCL do OLED para a placa na GP1
- VERMELHO: VCC do HC e OLED para a placa em 3.3v

- PRETO: GND do HC e OLED para a placa na GND

```
from machine import Pin, I2C
from ssd1306 import SSD1306_I2C
from oled import Write, GFX, SSD1306_I2C
from oled.fonts import ubuntu_mono_15, ubuntu_mono_20
import utime

trigger = Pin(3, Pin.OUT)
echo = Pin(2, Pin.IN)

def distancia():
  tempo = 0
  trigger.low()
  utime.sleep_us(2)
  trigger.high()
  utime.sleep_us(5)
  trigger.low()
  while echo.value() == 0:
  signaloff = utime.tick_us()
  while echo.value() == 1:
   signalon = utime.tick_us()
  tempo = signalon - signaloff
  return tempo

WIDTH = 128
HEIGHT = 64

i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
oled = SSD1306_I2C(WIDTH, HEIGHT, i2c)

write15 = Write(oled, ubuntu_mono_15)
write20 = Write(oled, ubuntu_mono_20)

while True:
  distancia_cm = (distancia() * 0.0343) / 2
  oled.fill(0)
  write20.text("Distância: ", 0, 0)
  write15.text(str(round(distancia_cm,1)) + " cm", 0, 20)
  oled.show()
```

5 Projetos com Sensores

5.1 Acender Lâmpada com LDR

Este projeto possui os seguintes materiais:

- Módulo LDR
- Módulo Relay
- Lâmpada, Bocal e Tomada (110/220v)

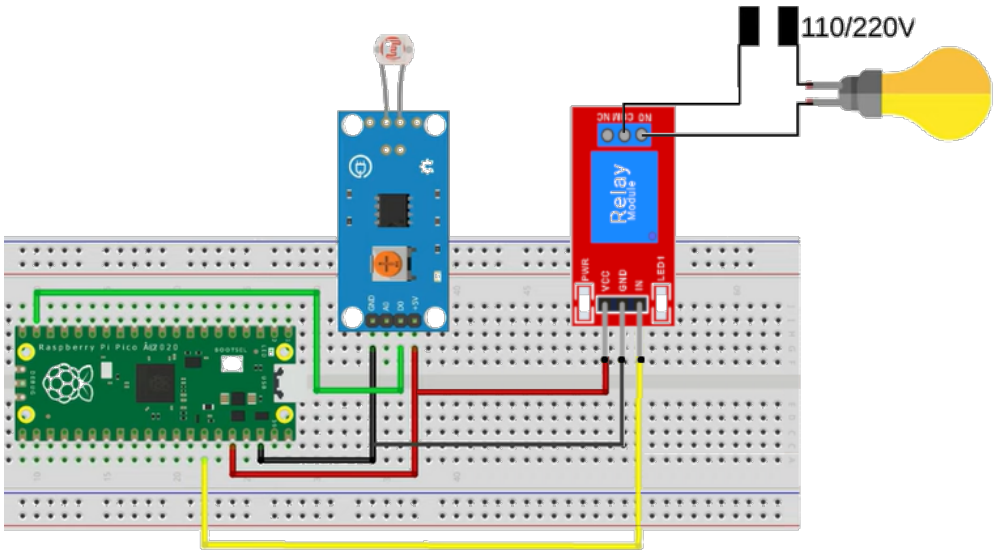


Figura 11: *Esquemático para o Projeto*

Ligações dos *jumpers*:

- AMARELO: IN do Relay para a placa na GP28
- VERDE: D0 do LDR para a placa na GP14
- VERMELHO: da placa em 3.3v
- PRETO: para a placa na GND
- Ligação 1 do RELAY em 110/220v: NC para a Lâmpada
- Ligação 2 do RELAY em 110/220v: COM para a Tomada

```
from machine import Pin
import time

relay = Pin(28, Pin.OUT)
ldr = Pin(14, Pin.IN, Pin.PULL_DOWN)

while True:
   if ldr.value():
      relay.value(1)
   else:
      relay.value(0)
   time.sleep(0.5)
```

5.2 Alarme com Sensor de Proximidade

Este projeto possui os seguintes materiais:

- Módulo Sensor de Movimento - PIR (HC-SR501)
- Sensor Buzzer (Passivo)

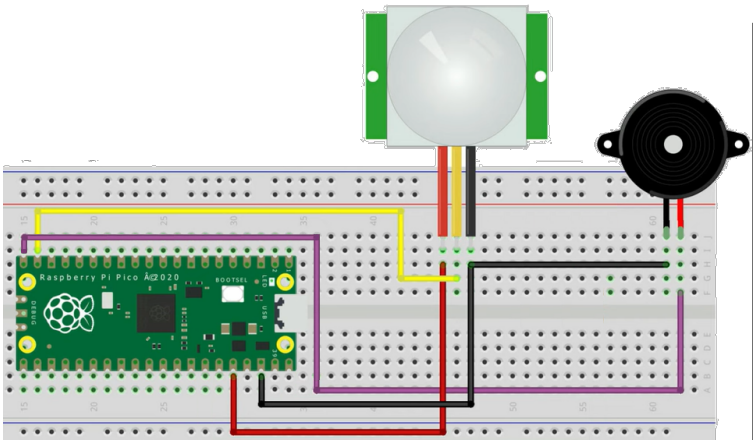


Figura 12: *Esquemático para o Projeto*

Ligações dos *jumpers*:

- AMARELO: PIR para a placa na GP14
- ROXO: Buzzer para a placa na GP15
- VERMELHO: da placa em 3.3v
- PRETO: para a placa na GND

```
1  from machine import Pin
2  import time
3
4  pir = Pin(14, Pin.IN, Pin.PULL_DOWN)
5  buzzer = Pin(15, Pin.OUT, Pin.PULL_DOWN)
6
7  while True:
8      if pir.value():
9          print("Movimento detectado")
10         buzzer.value(1)
11     else:
12         print("Nenhum movimento detectado")
13
14     time.sleep(1)
15     buzzer.value(0)
```

5.3 Umidade e Temperatura com Sensor DHT11

Este projeto possui os seguintes materiais:

- Módulo Sensor DHT11

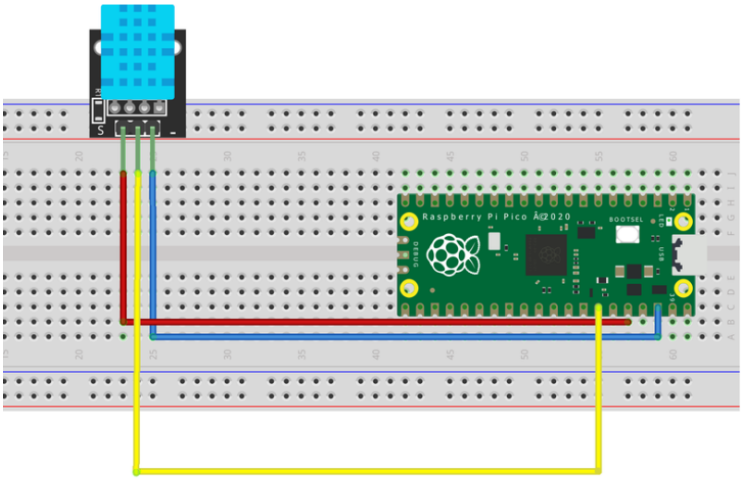


Figura 13: *Esquemático para o Projeto*

Ligações dos *jumpers*:

- ROXO: Buzzer para a placa na GP4
- VERMELHO: da placa em 3.3v
- PRETO: para a placa na GND

Primeiro precisamos disponibilizar a biblioteca **dht.py** na mesma pasta que estará o nosso programa.

```
from machine import Pin, I2C
import utime as time
from dht import DHT11, InvalidChecksum

while True:
  time.sleep(5)
  pin = Pin(28, Pin.OUT, Pin.PULL_DOWN)
  sensor = DHT11(pin)
  t  = (sensor.temperature)
  h = (sensor.humidity)
  print("Temperatura: {}".format(sensor.temperature))
  print("Umidade: {}".format(sensor.humidity))
```

5.4 Sensor de Chuva

Este projeto possui os seguintes materiais:

- Sensor de Chuva
- Módulo de Ajuste

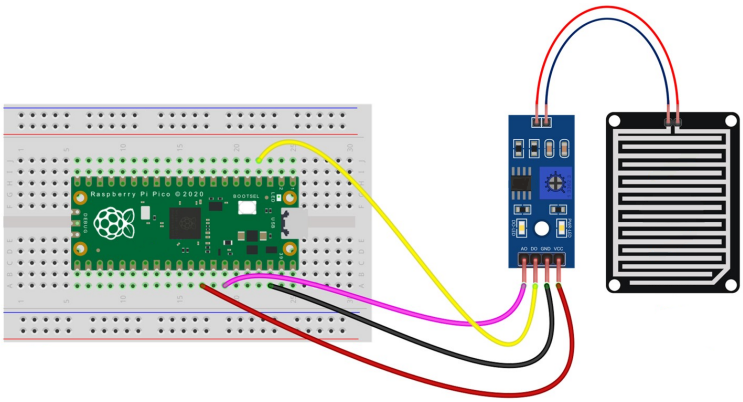


Figura 14: *Esquemático para o Projeto*

```
from machine import Pin
import utime

# Definicao dos Pinos
POWER_PIN = Pin(27, Pin.OUT)
DO_PIN = Pin(2, Pin.IN)

while True:
  // Liga o sensor
  POWER_PIN.value(1)
  // Espera 10 milisegundos
  utime.sleep_ms(10)
  // Obtem o valor
  rain_state = DO_PIN.value()
  // Desliga o sensor
  POWER_PIN.value(0)

  if rain_state == 1:
    print("Não está chovendo")
  else:
    print("Está chovendo")

  # Pausa de 1 segundo
  utime.sleep(1)
```

5.5 Sensor de Distância

Este projeto possui os seguintes materiais:

- Módulo Sensor Ultrassônico (HC-SR04)

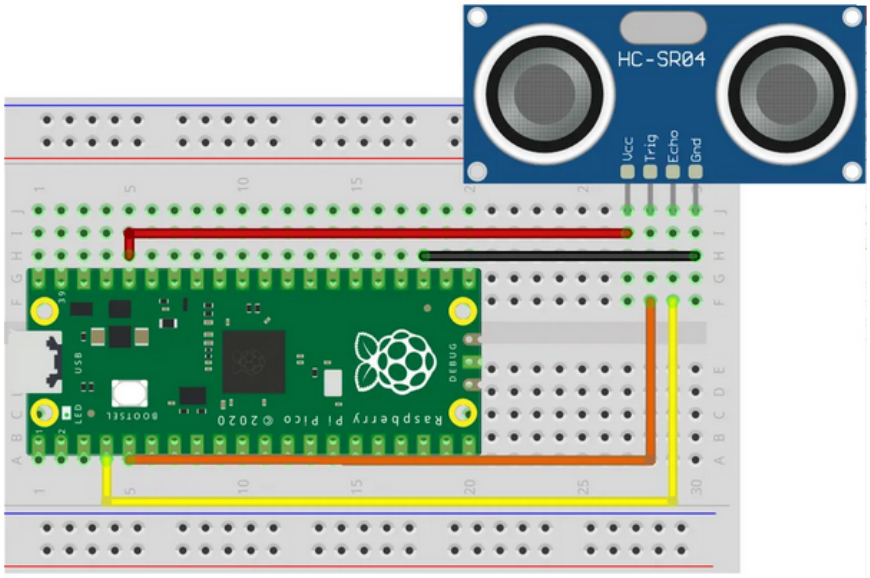


Figura 15: *Esquemático para o Projeto*

```
from machine import Pin
import utime

// Conexoes dos pinos
trigger = Pin(3, Pin.OUT)
echo = Pin(2, Pin.IN)

def distancia():
  // Desliga o envio da informacao
  trigger.low()
  // Espera 2 microsegundos
  utime.sleep_us(2)
  // Envia a informacao
  trigger.high()
  // Espera 5 microsegundos
  utime.sleep_us(5)
  // Desliga o envio da informacao
  trigger.low()
  // Espera o recebimento da informacao
  while echo.value() == 0:
    signaloff = utime.tick_us()
  while echo.value() == 1:
    signalon = utime.tick_us()
  // Calcula o tempo entre o envio e o recebimento
  tempo = signalon - signaloff
  return tempo

while True:
  // Velocidade do som 343 m/s ou 0.0343 cm/ms
  distancia_cm = (distancia() * 0.0343) / 2
  print("Distância: ", round(distancia_cm,1), "cm")
  utime.sleep(1)
```

5.6 Umidade e Temperatura com Sensor DS18B20

Este projeto possui os seguintes materiais:

- Módulo Sensor DS18B20

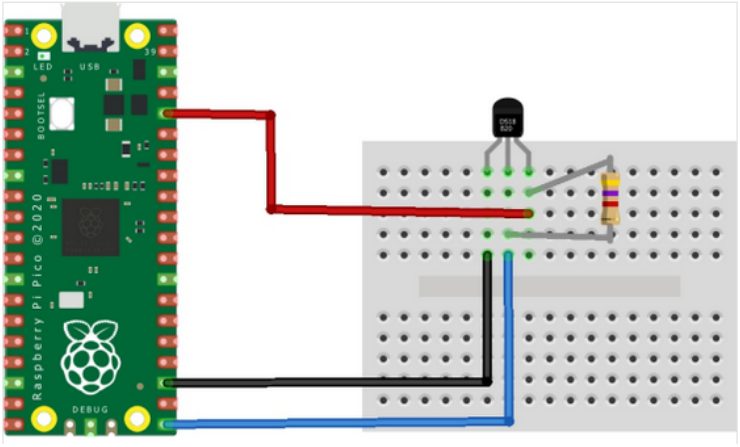


Figura 16: *Esquemático para o Projeto*

```
import machine, onewire, ds18x20, time

ds_pin = machine.Pin(16)
ds_sensor = ds18x20.DS18X20(onewire.OneWire(ds_pin))

roms = ds_sensor.scan()
print('Found a ds18x20 device')

while True:
  ds_sensor.convert_temp()
  time.sleep_ms(750)
  for rom in roms:
    print(ds_sensor.read_temp(rom))
  time.sleep(2)
```

5.7 Sensor IR

Este projeto possui os seguintes materiais:

- Módulo Sensor IR

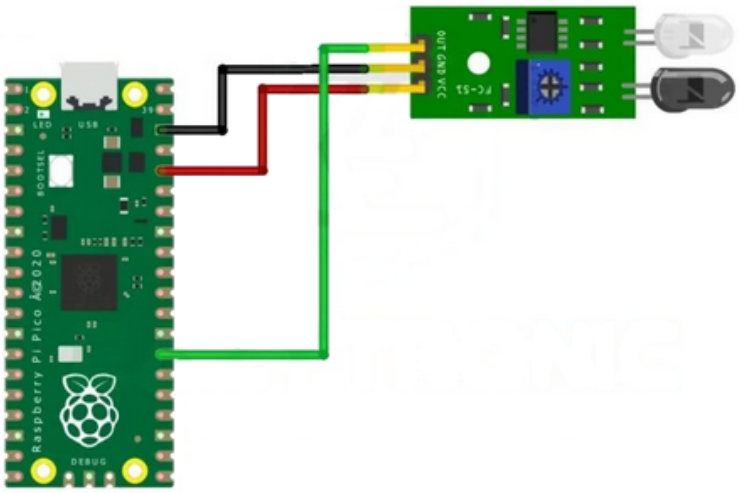


Figura 17: *Esquemático para o Projeto*

```
from machine import Pin
from utime import sleep_ms

// Ligacoes dos pinos
ir=Pin(21,Pin.IN)
led=Pin(25,Pin.OUT)

while True:
  try:
    if ir.value()==1:
      led.value(0)
    elif ir.value()==0:
      led.value(1)
    sleep_ms(50)
  except KeyboardInterrupt:
    break
```

6 Uso de Motores

6.1 Usar o Motor de Passo

Este projeto possui os seguintes materiais:

- Motor de Passo (28BYJ-48)
- Driver para motor (ULN2003)

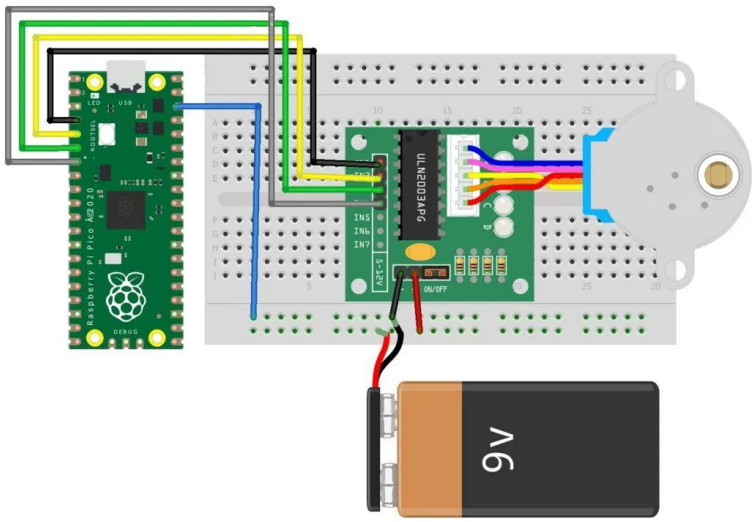


Figura 18: *Esquemático para o Projeto*

Ligações dos *jumpers*:

- PRETO: IN1 para a placa na GP2
- AMARELO: IN2 para a placa na GP3
- VERDE: IN3 para a placa na GP4
- CINZA: IN4 para a placa na GP5
- AZUL: da placa na GND

```
from machine import Pin
from time import sleep

IN1 = Pin(2,Pin.OUT)
IN2 = Pin(3,Pin.OUT)
IN3 = Pin(4,Pin.OUT)
IN4 = Pin(5,Pin.OUT)

pins = [IN1, IN2, IN3, IN4]

sequence = [[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1]]

while True:
   for step in sequence:
      for i in range(len(pins)):
         pins[i].value(step[i])
         sleep(0.001)
```

6.2 Servo Motor

Este projeto possui os seguintes materiais:

- Servo Motor (SG90)

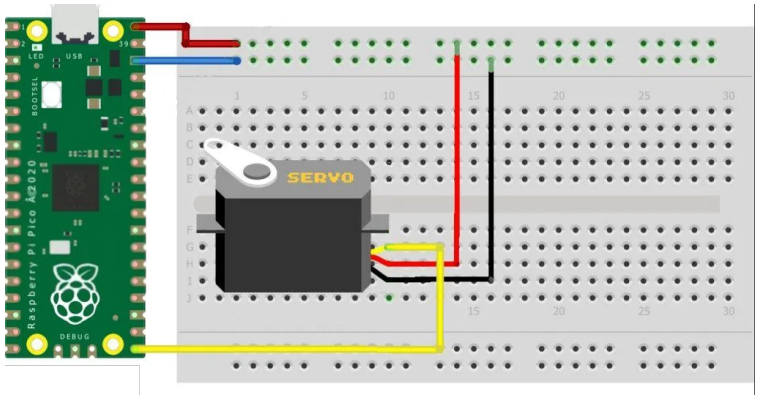


Figura 19: *Esquemático para o Projeto*

```
from machine import Pin, PWM
from time import sleep

servoPin = PWM(Pin(16))
servoPin.freq(50)

def servo(degrees):
  # Limite em graus entre 0 e 180
  if degrees > 180: degrees=180
  if degrees < 0: degrees=0
  # Maximo e minimo do ciclo de trabalho
  maxDuty=9000
  minDuty=1000
  # Novo ciclo entre o max e min na proporcao deste valor
  newDuty=minDuty+(maxDuty-minDuty)*(degrees/180)
  # PWM do valor do servo
  servoPin.duty_u16(int(newDuty))

  while True:
    # Inicia o Loop de 0 a 180 graus
    for degree in range(0,180,1):
      servo(degree)
      sleep(0.001)
      print("Incrementa:", degree, "graus")
    # Inicia o Loop de 180 a 0 graus
    for degree in range(180, 0, -1):
      servo(degree)
      sleep(0.001)
      print("Retrocede:", degree, "graus")
```

Explicação: O *duty cycle* representa quanto tempo o sinal fica em nível alto dentro de um período. Em outras palavras, define a largura do pulso do PWM.

- Duty baixo - pulso curto - menos energia / menor posição do servo
- Duty alto - pulso longo - mais energia / maior posição do servo
- minDuty = 1000 - largura de pulso mínima (posição inicial do servo, ex.: 0°)
- maxDuty = 9000 - largura de pulso máxima (posição final do servo, ex.: 180°)
- degrees - ângulo desejado do servo (de 0 a 180)

No código, estamos controlando um servo motor no Raspberry Pi Pico (MicroPython), usando duty_u16(), que recebe um valor de 0 a 65535 para definir o **PWM**. Quanto maior o *duty*, maior o

tempo em nível alto do sinal, e maior será o deslocamento do servo.

6.3 Motor DC com Ponte H

Este projeto possui os seguintes materiais:

- Motor DC
- Driver para motor Ponte H (L298D)

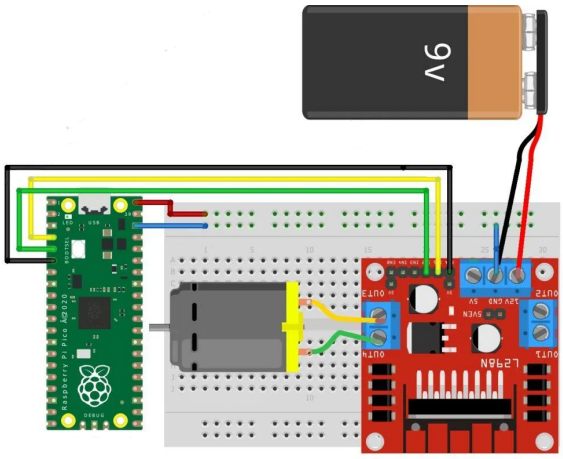


Figura 20: *Esquemático para o Projeto*

```
from machine import Pin
import utime

m1 = Pin(5, Pin.OUT)
m2 = Pin(4, Pin.OUT)

en1 = Pin(6, Pin.OUT)

# Habilita o motor
en1(1)

while True:
  # Avancar
  m1(1)
  m2(0)
  utime.sleep(1)
  # Parar
  m1(0)
  m2(0)
  utime.sleep(1)
  # Reverter
  m1(0)
  m2(1)
  utime.sleep(1)
```

7 Conclusão

O Raspberry Pi Pico é uma placa de microcontrolador notável por suas características robustas e custo acessível, rodando a linguagem de programação Python em sua variante MicroPython. Essa combinação oferece uma plataforma ideal para aprendizes, entusiastas e profissionais desenvolverem rapidamente protótipos funcionais.

Com a facilidade de interação em tempo real através do REPL, programadores podem testar e ajustar seu código instantaneamente, promovendo um ciclo de aprendizagem e desenvolvimento mais ágil. O amplo suporte de bibliotecas Python disponíveis acelera a integração com uma variedade de sensores e atuadores, expandindo o leque de possíveis aplicações práticas em áreas como automação residencial, robótica e controle ambiental.

A simplicidade e acessibilidade da linguagem Python, junto com as funcionalidades oferecidas pelo MicroPython, tornam o Raspberry Pi Pico uma ferramenta de ensino e desenvolvimento excepcional. Projetos como a automação de uma estufa ou a criação de um robô educativo se tornam muito mais tangíveis, permitem que mesmo programadores iniciantes possam se concentrar nos aspectos lógicos e criativos da interação com o hardware, sem as complicações de linguagens mais complexas.

Esta abordagem prática e orientada por exemplos facilita o aprendizado e a inovação, permite a transição rápida da teoria para a construção de projetos eletrônicos reais e complexos, satisfaz tanto a curiosidade quanto a necessidade de soluções práticas.

Sou um entusiasta do mundo **Open Source** e novas tecnologias. Qual a diferença entre Livre e Open Source? Livre significa que esta apostila é gratuita e pode ser compartilhada a vontade. Open Source além de livre todos os arquivos que permitem a geração desta (chamados de arquivos fontes) devem ser disponibilizados para que qualquer pessoa possa modificar ao seu prazer, gerar novas, complementar ou fazer o que quiser. Os fontes da apostila (que foi produzida com o LaTex) está disponibilizado no GitHub [3]. Veja ainda outros artigos que publico sobre tecnologia através do meu Blog Oficial [1].

Referências

[1] Fernando Anselmo - Blog Oficial de Tecnologia
`http://www.fernandoanselmo.blogspot.com.br/`

[2] Encontre essa e outras publicações em
`https://cetrex.academia.edu/FernandoAnselmo`

[3] Repositório para os fontes da apostila
`https://github.com/fernandoans/publicacoes`